

AI-Assisted Engineering

Practices that hold up when a machine writes the code — drawn from what is already working across our repositories.

The productivity question has been settled; the interesting question is **trust**. An agent will cheerfully produce a thousand lines that compile, pass, and are wrong. Everything below exists to make that failure mode visible in minutes rather than in production — and each item is running in a SingleComm repository today.

1 • Give the agent context, once, in the repository

The single highest-return artefact is a file that explains the codebase to whoever opens it next — human or otherwise.

Repository instructions One committed file describing structure, conventions and the rules that are **not** guessable. The ACD's is thirteen kilobytes and its most valuable line is a warning: two PHP generations live in this tree, and idioms must not cross between them. An agent cannot infer that; it will confidently write modern code into the legacy half.

Include the directory map, naming tables, build and validation commands, and the traps that have already cost someone a day.

Skills, one per domain Packaged domain knowledge an agent loads on demand instead of being re-taught each session. Ours live in a shared, version-controlled repository with its own linting — treated as code, reviewed as code. A skill is also how a tool becomes usable by an agent at all: both Understudy and the tower CLI ship one.

Decision records The most under-rated context there is. SingleConsole carries 27, and each states **the invariant its code must not break**. Handing an agent the decision beats any amount of prompt engineering, because it explains what would be wrong rather than what to type.

Contracts, not prose Where two implementations must agree, write the contract as a committed schema and generate the conformance suite from it. Our detection rules, status precedence and token shapes are pinned this way, and the same suite runs in PHP, TypeScript and Go — so "the CLI and the browser disagree" is a test failure, not a ticket.

2 • Shape the work so it can be finished

- **Ask for a plan before code** on anything non-trivial, and read it. Wrong assumptions are cheap to correct in a paragraph and expensive to correct in a branch.
- **One coherent outcome per task**. "Add the endpoint, migrate the schema, and refactor the caller" is three tasks; given as one, the weakest drags down the review of the other two.
- **Name the verification up front**. If you cannot say how the work will be checked, the agent will invent a check that passes.
- **Delegate exploration, keep the decisions**. Search and inventory suit a sub-agent working in parallel; architecture and trade-offs do not.
- **Run more than one at a time** on genuinely independent work — and resist it on anything touching the same files.

3 • The boundary: what stays human

The agent is good at

- Mechanical breadth — renames, migrations, test scaffolding
- Exploration and inventory across a large tree
- Applying a pattern that already exists, consistently

You still own

- What the invariant is, and what may never break
- Whether the abstraction earns its keep
- Anything touching money, identity or customer data
- The decision to ship

4 · Reviewing work you did not type

Reading the diff is necessary and nowhere near sufficient. These are the checks that have actually caught things, with what each one is for.

Gates, not reports	Coverage thresholds that fail the command rather than printing a number. SingleConsole holds 95% lines and 90% branches across 3,533 tests; Understudy ~97%. A threshold nobody enforces is a comment.
Mutation testing	The one to adopt first. Coverage tells you a line ran ; mutation asks whether anything breaks if that line is wrong . It found three real bugs in SingleConsole, the best of them a hand-rolled QR encoder emitting invalid codes that still looked exactly like QR codes — every structural test passed and no phone could read one. Not a pull-request gate: scope it to pure-logic packages where the tests are fast enough to run hundreds of times.
Derived checks	Checks that recompute a property rather than comparing to a snapshot. Our contrast validator re-derives every theme and preset token pair against the accessibility standard, and has caught eleven real regressions — including white call-to-action text on a bright background that looked fine to everyone who reviewed it.
Visual diffing	Shoot every state before and after, then generate a side-by-side gallery. Deterministic inputs make the two runs directly comparable, which is what makes a stray two-pixel shift visible. The gallery is self-contained, so it attaches to the pull request.
Dead-code hygiene	A scan for unused exports, files and dependencies, run as its own command. Generated code accretes orphans faster than hand-written code does, and nobody notices until the build is slow.
Two layers of test	A hermetic layer on every pull request — production build, real browser, no network, deterministic — and a live layer against a real cluster on a schedule. Matching identifiers across the two mean a failure in either names the same behaviour.
Watch it happen	For anything involving real-time media, an assertion is not enough. Run it headed and look. Understudy exists so that "I watched a synthetic customer talk to the console" is a thing you can do in one command, repeatably.

5 · Guardrails, so autonomy is safe rather than brave

- **Declare where a thing may run.** Every Understudy scenario names its permitted environments and forbids production by construction. This is the single change that makes letting an agent execute anything reasonable.
- **Never have an agent type a credential.** Capture a session once, interactively, and let the tooling reuse it. Secrets stay references, never values, and never reach a transcript.
- **Make the safe thing structural.** Our recorder cannot store a field its registry does not permit — unregistered types are dropped and counted. That is a stronger guarantee than a reviewer remembering to check, and it survives the next contributor.
- **Meaningful exit codes.** A tool an agent drives must distinguish "I could not reach it" from "I reached it and the answer is bad". Given one signal for both, an agent will confidently invent the difference.
- **Determinism where you can afford it.** One seeded generator means the same input produces the same world on any machine, forever — which is what makes a failure reproducible and a demo rehearsable.

6 · If you take four things away

1. **Write the repository instructions.** An afternoon, and it improves every session anyone has afterwards.
2. **Turn on mutation testing** for your pure-logic code and read the survivors. It is the fastest way to find out whether your tests assert anything.
3. **Say how the work will be verified** in the same breath as asking for it.
4. **Build the tool the agent needs**, and ship it with a skill. The leverage is not in using AI — it is in what you hand it.

Everything cited here is in a SingleComm repository and can be run today: repository instructions and skills in the ACD tree; decision records, contracts, mutation and contrast checks and the two-layer suite in SingleConsole; environment guards, baselines and headed runs in Understudy.